



INSTITUTO POLITECNICO NACIONAL ESCUELA SUPERIOR DE COMPUTO



Práctica 1: "Modelo Entidad Relación"

EJERCICIO 1

PARTE A

INVESTIGACION

Equipo:

Alonso Peña Valeria Elide

Perez Mateos Evelyn Yamilet

Martinez Gonzalez Irvin

Grupo: 3CV2

Profesora: Hurtado Aviles Gabriel

Septiembre 2026

Ejercicio 1. Control de versiones con Git y GitHub

Parte A. Investigación

1. ¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?

Los sistemas de control de versiones son software que ayudan a realizar un seguimiento de los cambios realizados en el código a lo largo del tiempo. Como desarrollador edita código, el sistema de control de versiones toma una instantánea de los archivos.

Sin el control de versiones, los desarrolladores tienen la tentación de mantener varias copias de código en su equipo. Esto es peligroso porque es fácil cambiar o eliminar un archivo en la copia incorrecta del código, lo que podría perder el trabajo. Los sistemas de control de versiones resuelven este problema administrando todas las versiones del código, pero presentando al equipo una sola versión a la vez.

El problema principal que resuelve es el pisotón o sobrescritura de archivos cuando dos o más personas intentan modificar el mismo documento o código al mismo tiempo.

Sin un sistema de control de versiones, ocurren situaciones como:

- Enviar archivos por correo o chat con nombres confusos (proyecto_final_v2_revisado_definitivo.docx).
- Perder el trabajo de un compañero porque otro usuario guardó una versión más vieja encima.
- No saber quién hizo un cambio específico ni por qué.

2. ¿Cuál es la diferencia entre Git y GitHub?

GitHub se basa en el software de código abierto, Git.

Git es un sistema de control de versiones que realiza un seguimiento de los cambios en los archivos. Le permite trabajar en su propia copia de un proyecto, registrar un historial de los cambios y combinar su trabajo con los cambios de otras personas de forma segura.

GitHub se basa en Git mediante el hospedaje de los proyectos de Git, denominados repositorios, en la nube, así como la adición de herramientas de planeación y colaboración que facilitan a los equipos trabajar juntos.

3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

Repositorio: Es la carpeta digital del proyecto donde se guardan todos los archivos, imágenes y código, junto con todo el historial de cambios realizados.

Ejemplo: Un archivo de google, donde todos los integrantes del equipo escriben algo y al mismo tiempo.

Confirmación (commit): Es un guardado formal de los avances que has realizado en el código.

Ejemplo: En Visual Studio Code sería un control S para guardar el avance del código y después ejecutarlo desde la terminal.

Rama (branch): Es una línea de trabajo independiente creada a partir de la versión principal del código.

Ejemplo: Quieres agregar la librería `#include <math.h>` para calcular una raíz cuadrada, pero no sabes si vas a romper el programa o si te va a dar errores al compilar con gcc.

Fusión (merge): Es el proceso de unir el trabajo de dos ramas distintas.

Ejemplo: Cuando un amigo une sus parte del trabajo.

Conflicto de fusión: Ocurre cuando dos personas modifican exactamente la misma línea de un mismo archivo.

Ejemplo: Cuando tú y tu amigo editan un documento de Google específicamente en la misma línea.

Pull Request: Es una petición formal que haces en una plataforma en la nube.

Ejemplo: Sería modo sugerencia en google docs.

Archivo .gitignore: Fichero de texto plano que especifica patrones de búsqueda.

Ejemplo: Los pasos de cómo compilar tu programa, qué hace el proyecto y quiénes son los autores.

Archivo README: Es la portada de presentación de cualquier proyecto.

Ejemplo: Al abrir un repositorio en GitHub, lo primero que ves abajo es el README.md

4. En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

Un flujo de trabajo basado en ramas (branching workflow) es una estrategia de organización en Git donde la rama principal (main) se mantiene siempre limpia, estable y lista para ejecutarse, mientras que cada nueva función, corrección de errores o experimento se desarrolla en su propia rama independiente.

BIBLIOGRAFÍAS

Atlassian. (s. f.). *¿Qué es el control de versiones?*

<https://www.atlassian.com/es/git/tutorials/what-is-version-control>

Jones, M. A. (2 de abril de 2026). *Git 01: Intro to Git version control*. NSF NEON.

<https://www.neonscience.org/resources/learning-hub/tutorials/intro-git-version-control>

Microsoft. (27 de octubre de 2025). *¿Qué es el control de versiones?* Microsoft Learn.

<https://learn.microsoft.com/es-es/devops/develop/git/what-is-version-control>